Lookahead-Then-Verify: Reliable Constrained Decoding for Diffusion LLMs under Context-Free Grammars

YITONG ZHANG, Tsinghua University, China
YONGMIN LI, Peking University, China
YUETONG LIU, Tsinghua University, China
JIA LI, Tsinghua University, China
XIAORAN JIA, Beijing Institute of Technology, China
ZHERUI LI, Nanyang Technological University, Singapore
GE LI, Peking University, China

arXiv:2602.00612v3 [cs.CL] 24 Aug 2026

Diffusion Large Language Models (dLLMs) have demonstrated promising capabilities and are increasingly used to produce formal languages defined by context-free grammars, such as source code and chemical expressions. However, as probabilistic models, they still struggle to generate syntactically valid outputs reliably. A natural and promising direction to address this issue is to adapt constrained decoding techniques to enforce grammatical correctness during generation. However, applying these techniques faces two primary obstacles. On the one hand, the non-autoregressive nature of dLLMs renders most existing constrained decoding approaches inapplicable. On the other hand, current approaches specifically designed for dLLMs may allow intermediate outputs that are impossible to complete into valid sentences, which significantly limits their reliability in practice.

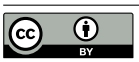
To address these challenges, we present *LAVE*, a constrained decoding approach specifically designed for dLLMs. Our approach leverages a key property of dLLMs, namely their ability to predict token distributions for all positions in parallel during each forward pass. Whenever a new token is proposed by the model, *LAVE* performs lookahead using these distributions to efficiently and reliably verify the validity of the proposed token. This design enforces reliable constraints by preserving the potential for intermediate outputs to be extended into valid sentences. Extensive experiments across four widely used dLLMs and five representative benchmarks demonstrate that *LAVE* consistently outperforms existing baselines and achieves improvements in syntactic correctness, while incurring negligible runtime overhead.

CCS Concepts: • **Computing methodologies** → **Natural language processing**; • **Software and its engineering** → **Syntax**; • **Theory of computation** → **Formal languages and automata theory**.

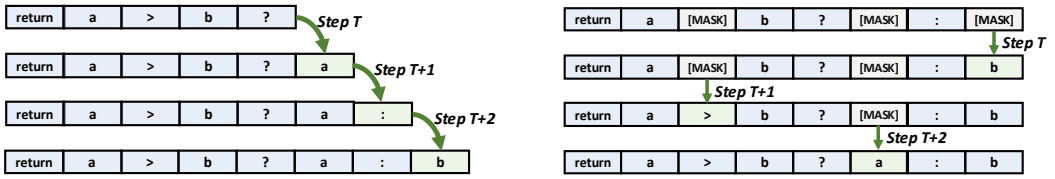
Additional Key Words and Phrases: Constrained Decoding, Diffusion Large Language Model, Trustworthy AI

ACM Reference Format:
Yitong Zhang, Yongmin Li, Yuetong Liu, Jia Li, Xiaoran Jia, Zherui Li, and Ge Li. 2026. Lookahead-Then-Verify: Reliable Constrained Decoding for Diffusion LLMs under Context-Free Grammars. *Proc. ACM Softw. Eng.* 3, ISSTA, Article ISSTA055 (October 2026), 23 pages. https://doi.org/10.1145/3832146

Authors' Contact Information: Yitong Zhang, Tsinghua University, College of AI, Beijing, China, 22373337@buaa.edu.cn; Yongmin Li, Peking University, School of Computer Science, Beijing, China, liyongmin@pku.edu.cn; Yuetong Liu, Tsinghua University, College of AI, Beijing, China, 23371522@buaa.edu.cn; Jia Li, Tsinghua University, College of AI, Beijing, China, jia_li@mail.tsinghua.edu.cn; Xiaoran Jia, Beijing Institute of Technology, School of Computer Science and Technology, Beijing, China, jiaxiaoran576@gmail.com; Zherui Li, Nanyang Technological University, College of Computing and Data Science, Singapore, Singapore, zhrli324@gmail.com; Ge Li, Peking University, School of Computer Science, Beijing, China, lige@pku.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2994-970X/2026/10-ARTISSTA055
https://doi.org/10.1145/3832146



(a) Autoregressive LLM (b) Diffusion LLM

Fig. 1. AR LLMs generate tokens autoregressively, so every intermediate output forms a *complete prefix*, whereas dLLMs generate tokens non-autoregressively, yielding intermediate outputs that are often *incomplete prefixes* due to remaining `[MASK]`.

1 Introduction

Recently, diffusion Large Language Models (dLLMs) [24, 25, 37] have attracted significant attention in both academia and industry due to their theoretical potential for high-efficiency generation [6, 21, 57]. Unlike AutoRegressive LLMs (AR LLMs), dLLMs generate tokens in a non-autoregressive manner, which allows tokens on the right to be generated before those on the left, as shown in Figure 1. Due to their promising capabilities, they are increasingly applied to tasks requiring the generation of formal languages, which are typically defined by a Context-Free Grammar (CFG) [38, 43]. For instance, a growing body of research has explored the potential of dLLMs in generating source code [15, 49] and chemical expressions [29]. However, as probabilistic models, ***dLLMs still struggle to generate syntactically valid outputs reliably in formal languages*** [39, 51], which limits their broader applicability in real-world scenarios. To assess the severity of this issue, we conducted a preliminary experiment on `HumanEval-CPP` [55]. Our results show that even leading dLLMs suffer from high syntax error rates. For example, `Dream-v0-Instruct-7B` [53] exhibits a syntax error rate of up to 23.8%.

To address this reliability issue, a promising direction is to adapt constrained decoding techniques, which have been widely used to ensure that AR LLMs produce syntactically valid outputs [43, 47, 48]. These approaches typically rely on grammar parsers [1, 8, 10] to guarantee the validity of each newly generated token [28, 44, 48]. Existing parsers are generally restricted to processing *complete prefixes* (*i.e.,* prefixes containing no ungenerated token), which naturally aligns with the autoregressive generation process of AR LLMs [8, 13, 38]. However, dLLMs generate tokens in a non-autoregressive manner. As shown in Figure 1b, dLLMs often produce *incomplete prefixes* (*i.e.,* prefixes containing some ungenerated tokens – `[MASK]`), which renders existing constrained decoding techniques inapplicable. Consequently, a key challenge lies in ***how to enable constrained decoding to function effectively on such incomplete prefixes***. Although recent work [29] has attempted to design CFG-based constrained decoding for dLLMs, it fails to ensure that the constraints are reliable. More concretely, it often leads to intermediate outputs that are impossible to complete into a valid sentence in the target language, which significantly undermines its reliability. Therefore, developing constrained decoding approaches tailored to dLLMs still remains an open problem.

To address this challenge, we propose *LAVE*, a constrained decoding approach specifically designed for dLLMs. At the core of our approach is a simple but powerful idea: ***lookahead-then-verify***. Unlike AR LLMs, which predict a single probability distribution for the next token, dLLMs predict distributions for all positions in parallel during every forward pass [12, 14, 51]. Leveraging this capability, when the model proposes a token for a masked position, *LAVE* can naturally perform lookahead by sampling tokens for the remaining masked positions within the current *incomplete prefix*. This process effectively transforms the *incomplete prefix* into a set of *complete prefixes* that reflect the likely continuations of the model and can be directly validated by most parsers [1, 8, 10]. Subsequently, *LAVE* employs a grammar parser to verify whether any of these lookahead *complete*

prefixes can be further extended into a valid sentence in the target language. The existence of such a prefix serves as a reliable guarantee that the proposed token can preserve the potential for the output to be completed into a valid sentence. If this condition is satisfied, the proposed token is accepted; otherwise, it is rejected and the model is required to propose another token.

However, implementing this idea presents two potential hurdles. ❶ First, an *incomplete prefix* may contain multiple masked positions, and the vocabulary size is typically very large. As a result, determining whether a proposed token should be accepted may in principle require exploring an exponential number of lookahead *complete prefixes*, which is computationally infeasible. Fortunately, our preliminary experiments in Section 3.2.2 show that sampling only a small number of lookahead prefixes is usually sufficient to decide whether a proposed token is acceptable, which reduces the runtime overhead of verification to a negligible level. ❷ Second, even with efficient verification, the iterative proposal and verification process may occasionally become trapped in difficult contexts, where the model repeatedly proposes tokens that fail verification. To address this issue, *LAVE* incorporates a lightweight recovery mechanism in Section 3.3 that actively adjusts the current context, allowing the model to escape such stalled states while preserving constraint reliability.

Our experimental results demonstrate that *LAVE* consistently outperforms all baselines across four widely used dLLMs and five representative benchmarks covering diverse formal languages. Across nearly all experimental settings, *LAVE* improves the syntactic correctness rate to almost 100%, enhancing the reliability of the model. Furthermore, in many scenarios, it yields improvements in functional correctness that are multiples of the gains achieved by other approaches. In terms of cost, we find that the runtime overhead introduced by *LAVE* is nearly negligible. We further conduct ablation studies, which show that each component contributes positively to the overall performance of *LAVE*.

In summary, our contributions are threefold:

- We identify an unreliability issue in existing approaches for constraining dLLMs under CFGs, where accepted tokens may render the output impossible to complete into a valid sentence.
- We propose *LAVE*, a constrained decoding approach that reliably enforces CFG constraints for dLLMs, ensuring that intermediate outputs remain syntactically extendable.
- We conduct extensive experiments using four widely used dLLMs. The results demonstrate that *LAVE* substantially improves the reliability of dLLMs in generating CFG-compliant outputs.

2 Background and Related Work

2.1 Diffusion Large Language Models

Diffusion LLMs have recently emerged as a promising alternative to traditional autoregressive LLMs, offering the potential for more efficient token prediction [21, 23, 36]. Recent advances have significantly narrowed the performance gap between dLLMs and AR LLMs, paving the way for the practical deployment of dLLMs in real-world scenarios. Early open-source dLLMs, such as LLaDA [32] and Dream [53], have demonstrated competitive performance comparable to LLaMA3-8B [40]. Furthermore, specialized models like Dream-Coder [49] and DiffuCoder [15] have been designed to excel in formal languages such as source code, achieving results on par with Qwen2.5-Coder-7B [18] across various code generation benchmarks. Mercury Coder [20] and Gemini Diffusion [7] have attained capabilities rivaling those of proprietary commercial AR LLMs while offering superior inference speeds.

The inference process of mainstream dLLMs follows an iterative denoising procedure in which a fully masked sequence $\mathbf{y}^0$ is gradually transformed into the final output. Formally, a dLLM f introduces a special `[MASK]` token and initializes generation with every position masked, denoted as $\mathbf{y}^0 = (\texttt{[MASK]})_{i=1}^{L}$, where L is the maximum generation length. Given a prompt $\mathbf{p}$, the model

performs up to T denoising steps to produce the final sequence $\mathbf{y}^T = (y_i^T)_{i=1}^L$, which contains no remaining `[MASK]` tokens. At each step $t \in \{1, \ldots, T\}$, the model performs a single forward pass to predict the token probability distributions $P_i(\cdot \mid \mathbf{p} \oplus \mathbf{y}^{t-1})$ in parallel for all positions i that are still filled with `[MASK]` tokens. A small subset of these masked positions is then decoded into non-`[MASK]` tokens according to the predicted distributions. Crucially, the generation order does not strictly follow a left-to-right order, which means that masks at later positions may be transformed into non-`[MASK]` tokens prior to those at earlier positions.

2.2 Constrained Decoding

Constrained decoding [3, 4, 13, 26, 27, 30, 34, 42, 43, 48, 54] is widely used to ensure that the output generated by LLMs adheres to a given grammar. However, most existing approaches in this line of work are designed for AR LLMs and rely on their left-to-right generation order. As a result, these approaches are not directly applicable to dLLMs, whose generation paradigm is inherently non-autoregressive. More recently, several studies have begun to explore constrained decoding tailored to dLLMs. For example, DINGO [39] introduces a dynamic programming-based decoding procedure that enables dLLMs to generate grammar-compliant outputs. However, this approach is limited to regular grammars, which restricts its applicability in realistic scenarios that typically require more expressive context-free grammars [10, 28].

The work most closely related to ours is that of Mündler et al. [29], who propose the first constrained decoding approach that enforces CFG compliance. Their approach adopts a classical rejection sampling strategy [19, 35] to impose grammatical constraints. The key idea is to reduce the validity check for a newly proposed token to a language intersection problem. To illustrate, consider a model generating C code with the current intermediate output `for ( [MASK] [MASK] . . .` . If the model proposes the token `)` for the second masked position, their approach constructs a regular language that accepts all strings matching the pattern $\texttt{for (}\ \Sigma^1\ \texttt{)}\ \Sigma^*$, where Σ denotes the vocabulary. This regular language is then intersected with the target context-free language, and the proposed token is accepted if and only if this intersection is non-empty.

Definition: Reliability of Constraints

In this paper, we formally define the reliability of constraints as follows [1].

- **Reliable Constraint:** At every decoding step, the intermediate output remains *extendable*, meaning that it can still be completed into a valid sentence in the target language.
- **Unreliable Constraint:** There exists at least one decoding step at which the intermediate output is *not extendable*, meaning that it can no longer be completed into any valid sentence in the target language.

Since constrained decoding aims to improve the reliability of LLM-generated outputs, enforcing reliable constraints is a fundamental requirement for practical deployment.

To ensure computational tractability, their approach has to construct the regular language by ignoring the actual length limit of masked spans, assuming that each masked span can be filled with an arbitrary number of tokens. In the example above, the regular language will be over-approximated as $\texttt{for (}\ \Sigma^*\ \texttt{)}\ \Sigma^*$ in practice. We argue that this over-approximation introduces constraint unreliability, as it may accept proposed tokens that render the intermediate output impossible to complete into a valid sequence. For instance, revisiting the example above, there is

[1]Following most prior work [3, 43, 48] on constrained decoding for AR LLMs, we define reliability without taking the maximum generation length into account.

only a single masked position between the parentheses, while a syntactically valid `for` statement requires at least two semicolons, which typically correspond to two tokens. Despite this requirement, the over-approximation allows the approach to accept the proposed token `)`, even though the resulting intermediate output can never be completed into a valid program.

Summary. Due to the non-autoregressive nature of dLLM generation, the vast majority of existing CFG-based constrained decoding approaches are rendered inapplicable. Moreover, the current approach tailored to dLLMs fails to enforce reliable constraints during inference, which substantially undermines its reliability. This highlights the need for a reliable constrained decoding approach tailored to dLLMs.

3 Methodology

In this section, we propose *LAVE*, a reliable constrained decoding approach designed for dLLMs. We first provide an overview of *LAVE* in Section 3.1, followed by detailed descriptions of its key components and implementation details in Section 3.2 and Section 3.3. Finally, we analyze the computational complexity of *LAVE* in Section 3.4.

3.1 Overview of *LAVE*

We constrain the decoding process of dLLMs through an iterative proposal and verification mechanism, as illustrated in Algorithm 1. During inference, the dLLM is allowed to propose new tokens for any masked positions according to its native decoding strategy (Line 5 in Algorithm 1). Each proposed token updates the current output by replacing a `[MASK]` token with a non-`[MASK]` token (Line 6 in Algorithm 1). The effectiveness of this decoding process critically depends on how the validity of each proposed token is assessed. To this end, we apply **Lookahead-Based Verification** (Section 3.2), the core component of *LAVE*, to reliably determine whether a proposed token should be accepted or rejected (Lines 7–13 in Algorithm 1). If the token is accepted, the dLLM proceeds to generate the next token; otherwise, it attempts to propose an alternative.

In certain cases, the model may repeatedly fail to produce an acceptable proposal [19, 27, 29, 41]. To mitigate this issue, we track the number of consecutive failed proposal attempts (Lines 10 and 12) and invoke **Cache-Enhanced Recovery** (Section 3.3) once this number reaches a predefined threshold (Lines 14–17 in Algorithm 1), enabling the model to recover from stalled inference states.

This process continues until no `[MASK]` token remains (Line 4 in Algorithm 1), after which *LAVE* returns the final output (Lines 18–19 in Algorithm 1).

3.2 Lookahead-Based Verification

In this section, we introduce Lookahead-Based Verification, which is the core of *LAVE*. Figure 2 illustrates how verification is performed.

When the model proposes a new token, it updates the current output by replacing a specific `[MASK]` token with a non-`[MASK]` token. The goal of Lookahead-Based Verification is to reliably verify whether this update preserves the potential for the output to be completed into a valid sentence.

3.2.1 Problem Formulation. Formally, let $\mathbf{y} \in (\Sigma \cup \{\texttt{[MASK]}\})^L$ denote the updated output that includes the newly proposed token. We define the index of the rightmost non-`[MASK]` token as

$$r(\mathbf{y}) = \max\{\, i \mid y_i \neq \texttt{[MASK]} \,\}, \tag{1}$$

where $r(\mathbf{y}) = 0$ if all tokens are masked. This index determines a prefix $\mathbf{y}^p = \mathbf{y}_{1:r(\mathbf{y})}$, which is typically an *incomplete prefix* containing several `[MASK]` tokens. We denote the set of masked

Algorithm 1 *LAVE*

Require: dLLM model f, Context-Free Grammar $\mathcal{G}$, prompt $\mathbf{p}$, lookahead size N, attempt budget τ

Initialize $\mathbf{y}$ with `[MASK]` tokens
Initialize counter $c_{\text{fail}} \leftarrow 0$
Initialize $\mathbf{y}_{\text{cache}} \leftarrow \epsilon$
while there are remaining `[MASK]` tokens **do**
  Propose a token t^* for a masked position i^* using f
  Create a temporary updated output $\mathbf{y}' \leftarrow \mathbf{y}$ with $\mathbf{y}'_{i^*} \leftarrow t^*$
  if Lookahead-Based Verification($\mathbf{y}'$) == `true` **then** ▷ ***Section 3.2***
    $\mathbf{y} \leftarrow \mathbf{y}'$ ▷ *Accept the proposed token*
    Update $\mathbf{y}_{\text{cache}}$ with a verified *complete prefix*
    $c_{\text{fail}} \leftarrow 0$
  else
    $c_{\text{fail}} \leftarrow c_{\text{fail}} + 1$ ▷ *Reject the proposed token*
  end if
  if $c_{\text{fail}} \geq \tau$ **then** ▷ ***Section 3.3***
    Apply **Cache-Enhanced Recovery** to $\mathbf{y}$ with $\mathbf{y}_{\text{cache}}$
    $c_{\text{fail}} \leftarrow 0$
  end if
end while
Output: final output $\mathbf{y}$ without `[MASK]` tokens

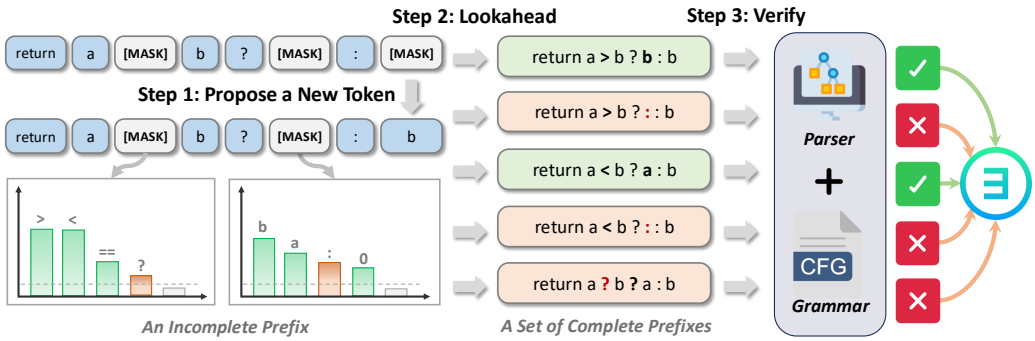


Fig. 2. The dLLM proposes a token for one masked position, which is subsequently verified through Lookahead-Based Verification. A proposed token is accepted if at least one lookahead *complete prefix* is extendable under the target grammar; otherwise, it is rejected.

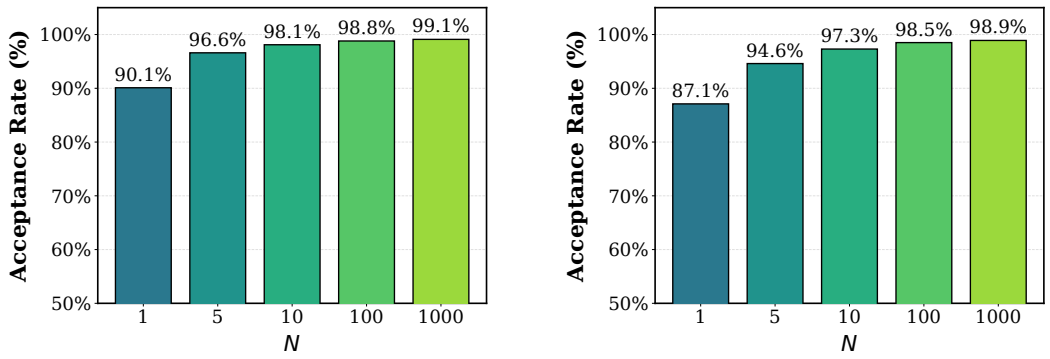
positions inside this prefix as

$$\mathcal{M}(\mathbf{y}^p) = \{\, i \mid 1 \leq i < r(\mathbf{y}),\ y_i^p = \texttt{[MASK]} \,\}. \tag{2}$$

A proposed token can be reliably accepted as long as there exists at least one assignment of tokens to these remaining masked positions such that the resulting *complete prefix* is a valid prefix:

$$\exists \mathbf{z} \in \Sigma^{|\mathcal{M}(\mathbf{y}^p)|} \quad \text{s.t.} \quad \text{IsExtendable}\big(\Phi(\mathbf{y}^p, \mathbf{z})\big) = \texttt{true}, \tag{3}$$

where $\Phi(\cdot,\cdot)$ is a function that fills the masked positions in $\mathbf{y}^p$ using the assignment $\mathbf{z}$. The function $\text{IsExtendable}(\cdot)$ is a standard capability provided by most grammar parsers (*e.g.,* Earley [10] and LR [1] parsers) and can efficiently check whether a *complete prefix* can be extended into a valid sentence.



(a) Dream-v0-Instruct-7B (b) LLaDA-8B-Instruct

Fig. 3. Experimental results from our preliminary empirical study.

3.2.2 Motivation and Feasibility. dLLMs offer a distinct advantage over AR LLMs: they predict token distributions for all positions simultaneously. This property enables a natural *lookahead-then-verify* strategy. Given an *incomplete prefix* $\mathbf{y}^p$, we can perform lookahead by filling the masked positions in $\mathcal{M}(\mathbf{y}^p)$ according to the model's predicted distributions, thereby transforming $\mathbf{y}^p$ into a set of *complete prefixes* that reflect the model's likely continuations. These complete prefixes can then be checked using a grammar parser via IsExtendable$(\cdot)$, allowing us to verify whether the current output reliably remains extendable.

The effectiveness and efficiency of this strategy depend on how many *complete prefixes* need to be sampled. While sampling a larger number of *complete prefixes* increases the likelihood of acceptance, it may also introduce non-negligible overhead. To assess this trade-off and validate the feasibility of our approach, we conducted an empirical study.

We evaluated two widely used models, Dream-v0-Instruct-7B [53] and LLaDA-8B-Instruct [32], on the `HumanEval-CPP` benchmark [55]. For each test instance, we randomly selected a prefix from the reference solution and used it to prefill the output, thereby forming a *complete prefix*. To simulate an intermediate inference state, we then randomly masked tokens within this *complete prefix* to obtain an *incomplete prefix*, which was guaranteed to be extendable. Specifically, each token was retained with a probability of 80% and replaced with `[MASK]` otherwise. We then performed a single forward pass to obtain the predicted probability distributions at each masked position. To test the feasibility of our approach, we sampled N distinct *complete prefixes* from these distributions and checked whether any of them could be extended into a valid sentence under the target CFG. A higher acceptance rate indicates that our motivation is practically feasible in reliably determining the extendability of *incomplete prefixes*. As illustrated in Figure 3, we observe that sampling only a small number of *complete prefixes* is typically sufficient to uncover a valid *complete prefix* whenever the underlying *incomplete prefix* has the potential to be extended into one valid sentence. In particular, with a lookahead size of $N = 10$, the acceptance rate reaches 98.1% for `Dream-v0-Instruct-7B` and 97.3% for `LLaDA-8B-Instruct`. These results indicate that the proposed *lookahead-then-verify* strategy is feasible in practice.

3.2.3 Verification Procedure. Based on the feasibility of the proposed *lookahead-then-verify* strategy, we now describe how it is implemented in practice.

Let $P_j(\cdot)$ denote the predicted token distribution at each masked position $j \in \mathcal{M}(\mathbf{y}^p)$. To perform **lookahead**, we sample N distinct assignments $\{\mathbf{z}^{(1)}, \ldots, \mathbf{z}^{(N)}\}$ according to these marginal

distributions $P_j(\cdot)$. Formally, each assignment $\mathbf{z}$ is sampled from the joint distribution:

$$P(\mathbf{z}) = \prod_{j \in \mathcal{M}(\mathbf{y}^p)} P_j(z_j). \tag{4}$$

Each sampled assignment $\mathbf{z}^{(n)}$ is then used to fill all masked positions in $\mathbf{y}^p$, resulting in a *complete prefix*:

$$\mathbf{s}^{(n)} = \Phi(\mathbf{y}^p, \mathbf{z}^{(n)}), n \in \{1, \cdots, N\}, \tag{5}$$

which no longer contains any `[MASK]` tokens.

In the **verification** stage, we apply grammar parsers to each *complete prefix* to determine whether it is extendable in parallel. The proposed token is accepted if and only if at least one *complete prefix* satisfies:

$$\exists n \in \{1, \ldots, N\} \quad \text{s.t.} \quad \mathsf{IsExtendable}(\mathbf{s}^{(n)}) = \mathtt{true}. \tag{6}$$

Such a *complete prefix* serves as a constructive witness that the updated output remains grammatically extendable, thereby naturally ensuring the reliability of the constraint. Conversely, if no valid *complete prefix* is found after N lookahead attempts, the proposed token is rejected, and another proposed token is sampled from the model's original predicted distribution.

3.3 Cache-Enhanced Recovery

In practice, the iterative proposal and verification process may occasionally enter a challenging state in which the current context makes it difficult for the model to produce an acceptable token. Concretely, the model may repeatedly propose tokens that fail the Lookahead-Based Verification, causing the decoding process to nearly stall. To address this issue, we introduce a recovery strategy that **actively modifies the current context**.

Our recovery strategy leverages a byproduct of the Lookahead-Based Verification process described in Section 3.2. Specifically, whenever a *complete prefix* $\mathbf{s}^{(n)}$ passes the verification, it constitutes a valid prefix that is both syntactically extendable and favored by the model. We record such prefixes by maintaining a global variable $\mathbf{y}_{\text{cache}}$, which is updated whenever a verification succeeds:

$$\mathbf{y}_{\text{cache}} \leftarrow \mathbf{s}^{(n)}, \quad \text{if } \mathsf{IsExtendable}(\mathbf{s}^{(n)}) = \mathtt{true}. \tag{7}$$

During inference, we monitor the number of consecutive failed proposal attempts. When this count reaches a predefined proposal budget τ, we infer that the model is trapped in a difficult context. To break this impasse, we replace the prefix of the current output with $\mathbf{y}_{\text{cache}}$:

$$\mathbf{y}_{1:r(\mathbf{y})} \leftarrow \mathbf{y}_{\text{cache}}. \tag{8}$$

However, when $\mathbf{y}_{1:r(\mathbf{y})}$ contains no remaining `[MASK]` tokens (*i.e.,* $\mathbf{y}_{1:r(\mathbf{y})}$ is identical to $\mathbf{y}_{\text{cache}}$), this replacement alone does not change the current context. To ensure that the context can be effectively modified, we generate one additional token following the current prefix. Many parsers used in existing constrained decoding approaches [4, 28, 44] can efficiently compute the set of valid successor tokens given a *complete prefix*. All tokens in this set are guaranteed to preserve the syntactic validity of the prefix. Let $\mathcal{V} = \mathsf{NextTokens}(\mathbf{y}_{\text{cache}})$ denote the set of such admissible tokens. We define an adjusted probability distribution for the next position as follows:

$$\hat{P}_{r(\mathbf{y})+1}(t) \propto P_{r(\mathbf{y})+1}(t)\, \mathbb{I}[t \in \mathcal{V}]. \tag{9}$$

We then sample the next token from $\hat{P}_{r(\mathbf{y})+1}$ and continue inference with the standard proposal and verification process, thereby preserving both the non-autoregressive generation behavior and the constraint reliability of subsequent generation.

3.4 Complexity Analysis

According to Algorithm 1, each decoding step allows at most τ failed proposals, and each proposal requires at most N lookahead verifications. With an Earley parser [10], verifying a prefix of length ℓ takes $O(\ell^3)$ time for a general CFG. Therefore, the additional cost of generating one accepted token is bounded by $O(\tau N \ell^3)$. For practical deterministic CFGs, this cost reduces to $O(\tau N \ell)$.

4 Experimental Setup

In this section, we introduce the experimental setup, including the benchmarks and their corresponding context-free grammars, the compared baselines, the evaluation metrics, the models, and other implementation details. Based on this setup, we conduct a series of experiments designed to address the following Research Questions (RQs).

- **RQ1**: How well does *LAVE* ensure the syntactic correctness of dLLM outputs compared to baseline approaches?
- **RQ2**: How well does *LAVE* improve the functional correctness of dLLM outputs?
- **RQ3**: What runtime overhead does *LAVE* introduce?
- **RQ4**: How does each component of *LAVE* contribute to the overall performance?
- **RQ5**: How do the hyperparameters affect the performance of *LAVE*?

4.1 Benchmark

To ensure a fair comparison, we follow existing studies [29] in selecting benchmarks for evaluation. Specifically, we adopt five benchmarks that require the model to generate representative formal languages defined by context-free grammars.

- **Code Generation:** To evaluate the generalizability of our approach across different programming languages, we consider three code generation benchmarks derived from HumanEval [5], namely **HumanEval-CPP**, **HumanEval-Java**, and **HumanEval-Go** [55], which are designed to evaluate the code generation ability of LLMs in C++, Java, and Go, respectively. Each benchmark contains 164 problems. For each problem, the dataset provides a natural language problem description and a set of test cases to verify the correctness of the generated output. We include these benchmarks to demonstrate the performance of *LAVE* in code generation scenarios across multiple programming languages. In the following sections, we refer to these benchmarks as `CPP-Bench`, `JAVA-Bench`, and `GO-Bench`, respectively.
- **JSON Generation: JSON-Mode-Eval-Extended [29]** is constructed based on the JSON-Mode-Eval benchmark [33] to evaluate the information extraction capabilities of LLMs using the JSON format. The benchmark consists of 272 problems. Each problem comprises a natural language snippet, a JSON Schema (corresponding to a specific CFG), and a ground truth JSON response. We include this benchmark because it can effectively reflect the performance of *LAVE* in generating JSON, a formal language that is widely employed in information extraction and agent function calling tasks. In the following sections, we refer to this benchmark as `JSON-Bench`.
- **Chemical Expression Generation: SMILES-Eval [29]** is constructed to assess LLM capabilities in generating SMILES chemical molecule representations from natural language descriptions. The benchmark consists of 167 problems. Each problem includes a natural language molecule description and a corresponding SMILES string as the ground truth. We include this benchmark to highlight the applicability of *LAVE* in scientific domains. In the following sections, we refer to this benchmark as `SMILES-Bench`.

4.2 Grammar

For CFG-constrained decoding, a concrete grammar specification is required. For `CPP-Bench`, `JAVA-Bench`, `GO-Bench`, and `SMILES-Bench`, we construct grammars based on the standard syntax of C++, Java, Go, and SMILES, respectively. For `JSON-Bench`, where each problem is associated with a distinct JSON Schema, we follow prior work [29] and automatically construct a corresponding grammar from the provided JSON Schema for each problem.

4.3 Baselines

We compare *LAVE* against three baseline approaches. ❶ First, we include the constrained decoding approach proposed by Mündler et al. [29], which we refer to as IG-CD (Intersection-Guided Constrained Decoding). ❷ Second, we include the unconstrained generation setting, denoted as NO-CD. ❸ In addition, we design a straightforward constrained decoding baseline that enforces a strictly left-to-right generation order for dLLMs, thereby allowing the direct application of constrained decoding techniques [28] originally developed for AR LLMs. We refer to this baseline as FS-CD (Forced-Sequential Constrained Decoding).

4.4 Metrics

We evaluate all approaches using three metrics: *syntactic@k*, *functional@k*, and average inference time.

❶ ***syntactic@k*** measures the proportion of problems for which **at least one** of the k generated outputs satisfies the syntactic constraints specified by the given context-free grammar.

❷ ***functional@k*** measures the proportion of problems for which **at least one** of the k generated outputs exhibits correct functional behavior. For `CPP-Bench`, `GO-Bench`, and `JAVA-Bench`, functional correctness is determined by executing the provided test cases. For the other benchmarks, it is assessed by exact matching against the ground-truth outputs. In general, syntactic correctness is a prerequisite for functional correctness.

❸ **Average Inference Time** records the average decoding time per generation, providing a measure of the overhead introduced by different constrained decoding approaches.

4.5 Models

We evaluate our approach using four widely used dLLMs:

- `LLaDA-8B-Instruct` [32] is released by researchers from Renmin University of China. It is an early open-source diffusion large language model with 8 billion parameters and is trained entirely from scratch. In the following, we refer to this model as `LLaDA-8B`.
- `LLaDA-1.5` [56] is also released by researchers from Renmin University of China. It extends `LLaDA-8B-Instruct` by incorporating Variance Reduced Preference Optimization (VRPO) during training. Compared with `LLaDA-8B-Instruct`, it achieves improved performance across a wide range of tasks, including code-related tasks.
- `Dream-v0-Instruct-7B` [53] is released by researchers from The University of Hong Kong. It is initialized from an autoregressive language model and demonstrates strong overall capabilities. In the following, we refer to this model as `Dream-7B`.
- `DiffuCoder-7B-cpGRPO` [15] is released by researchers from Apple and The University of Hong Kong. It is post-trained with coupled-GRPO on 21K code examples, leading to substantial improvements in code generation performance. In the following, we refer to this model as `DiffuC-7B`.

Table 1. *syntactic@k* (%) on JSON-Bench, CPP-Bench, JAVA-Bench, GO-Bench, and SMILES-Bench, where $k \in \{1, 5, 10\}$. The **best result** for each model, benchmark, and k is highlighted in bold.

Method	JSON-Bench			CPP-Bench			JAVA-Bench			GO-Bench			SMILES-Bench		
	k=1	k=5	k=10	k=1	k=5	k=10	k=1	k=5	k=10	k=1	k=5	k=10	k=1	k=5	k=10
LLaDA-8B															
NO-CD	85.4	90.1	96.4	74.2	94.1	95.6	72.0	86.6	89.6	72.0	86.1	87.5	66.0	75.9	78.9
FS-CD	94.9	96.4	96.4	66.3	66.7	66.7	78.7	79.4	79.4	81.7	82.2	82.3	75.7	75.9	76.1
IG-CD	94.8	95.7	96.7	86.1	96.7	98.1	–	–	–	–	–	–	94.3	98.7	99.2
LAVE	**99.5**	**99.6**	**99.6**	**90.9**	**99.3**	**99.4**	**92.1**	**94.5**	**95.1**	**91.5**	**97.7**	**99.4**	**98.7**	**100.0**	**100.0**
LLaDA-1.5															
NO-CD	86.0	87.7	87.9	78.2	92.0	94.8	78.1	87.2	90.9	55.5	71.3	76.8	62.3	77.7	83.1
FS-CD	96.0	97.7	98.1	73.8	74.2	74.2	78.1	78.1	78.1	77.4	82.3	83.5	78.3	78.5	78.5
IG-CD	94.0	95.4	96.2	89.5	97.3	**98.8**	–	–	–	–	–	–	93.7	97.0	97.6
LAVE	**99.7**	**99.8**	**99.8**	**91.3**	**98.8**	**98.8**	**91.5**	**94.5**	**95.1**	**89.6**	**94.5**	**97.0**	**96.6**	**100.0**	**100.0**
Dream-7B															
NO-CD	86.8	89.3	89.3	76.2	79.2	81.1	69.5	70.1	70.1	64.6	70.7	72.0	74.0	76.2	76.8
FS-CD	96.8	98.2	98.2	83.1	84.4	84.4	84.8	85.1	85.1	78.4	78.7	78.7	93.1	93.8	93.8
IG-CD	94.9	95.5	95.6	87.6	90.4	90.9	–	–	–	–	–	–	95.1	96.7	97.4
LAVE	**99.0**	**99.6**	**99.7**	**96.0**	**99.4**	**99.4**	**90.9**	**93.3**	**94.5**	**89.3**	**96.3**	**98.2**	**99.4**	**100.0**	**100.0**
DiffuC-7B															
NO-CD	89.9	90.4	90.4	79.9	80.5	81.7	82.3	82.9	83.5	78.7	79.3	79.9	70.1	71.3	71.3
FS-CD	96.9	98.1	98.1	93.8	94.0	94.3	91.5	91.5	91.5	92.1	92.5	92.7	96.0	96.3	96.3
IG-CD	92.1	92.7	92.7	93.8	95.1	95.3	–	–	–	–	–	–	96.4	96.6	96.7
LAVE	**99.6**	**100.0**	**100.0**	**97.0**	**99.6**	**100.0**	**93.3**	**95.5**	**95.7**	**96.5**	**99.1**	**99.4**	**98.8**	**99.3**	**99.4**

4.6 Implementation Details

Following prior studies [29, 51], we apply a simple and commonly used optimization for all approaches: once a dLLM generates an [EOS] token, all subsequent token positions are set to [EOS] to accelerate inference. For all experiments, we adopt the widely used semi-autoregressive strategy [2, 32, 45, 46, 52] and set the block size to 32, as is common in prior studies [17, 31, 56]. Additionally, for each experiment, we conduct at least four independent runs and report the average values of the evaluation metrics. For both FS-CD and *LAVE*, we employ the Earley algorithm [10] to implement the required grammar parser, which allows us to handle arbitrary context-free grammars. All experiments are conducted on a server equipped with eight NVIDIA A100 GPUs, each with 40 GB of memory, and two Intel Xeon Gold 6348 CPUs, with 28 cores per socket and 56 physical cores in total.

Regarding the hyperparameters of the dLLMs, we set the maximum generation length L to 256. The number of denoising steps T is set to 128, which provides a balanced trade-off between model effectiveness and efficiency [21, 32]. The temperature is fixed to 0.2 for all experiments. Regarding the hyperparameters specific to *LAVE*, we set the lookahead size N defined in Section 3.2 to 10 and the attempt budget τ defined in Section 3.3 to 5.

5 Experimental Results

5.1 RQ1: How Well Does *LAVE* Ensure the Syntactic Correctness of dLLM Outputs?

Motivation. The primary objective of *LAVE* is to ensure that the outputs generated by dLLMs reliably conform to the syntax defined by a CFG. To assess this capability, we compare the syntactic correctness of the outputs generated by our approach with that of the baseline outputs in this RQ.

Setting. We apply the baselines and *LAVE* to the four models described in Section 4.5 and evaluate their performance on the five benchmarks introduced in Section 4.1. We use *syntactic@k* as the

evaluation metric with $k \in \{1, 5, 10\}$, as defined in Section 4.4. Unfortunately, we cannot evaluate IG-CD on `GO-Bench` or `JAVA-Bench`, because the original paper [29] does not provide IG-CD-compatible grammars for Go and Java, nor does it publicly release the grammar construction rules used by IG-CD. Therefore, we cannot faithfully reproduce IG-CD on these two benchmarks.

Results. The *syntactic@k* scores of different approaches are reported in Table 1.

***LAVE* demonstrates superior performance in ensuring that dLLM outputs conform to CFG constraints.** Compared to the unconstrained baseline, *LAVE* substantially improves syntactic correctness, reaching close to 100% in most settings. For example, on the syntactically simpler `SMILES-Bench` with `LLaDA-8B`, *LAVE* raises *syntactic@1* from 66.0% to 98.7%, *syntactic@5* from 75.9% to 100.0%, and *syntactic@10* from 78.9% to 100.0%. On the more syntactically complex `CPP-Bench`, our approach also achieves strong performance. With `DiffuC-7B`, *LAVE* increases *syntactic@1* from 79.9% to 97.0%, *syntactic@5* from 80.5% to 99.6%, and *syntactic@10* from 81.7% to 100.0%. We further observe similarly strong improvements on the relatively lower-resource `GO-Bench`. For example, *LAVE* raises *syntactic@10* from 87.5% to 99.4% on `LLaDA-8B`, and from 72.0% to 98.2% on `Dream-7B`. The only exception is `JAVA-Bench`. Although *LAVE* still yields substantial improvements on this benchmark, its syntactic correctness remains harder to push to nearly 100%. By examining representative cases, we find that this limitation mainly arises because Java programs are typically more verbose, making the max generation length of 256 occasionally insufficient, rather than reflecting a weakness of our approach itself. These results demonstrate that *LAVE* greatly enhances the reliability of dLLM outputs when generating formal languages such as C++, Java, Go, JSON, and SMILES.

***LAVE* outperforms existing constrained decoding approaches across most evaluated benchmarks and models.** For example, on `JSON-Bench` with `LLaDA-8B`, *LAVE* improves *syntactic@1* over the best baseline FS-CD by 4.6%. On `CPP-Bench` with `Dream-7B`, *LAVE* outperforms the best baseline IG-CD by 8.4% in terms of *syntactic@1*. In addition, we observe that *LAVE* benefits more from increasing k than FS-CD. For instance, on `CPP-Bench` with `LLaDA-1.5`, as k increases from 1 to 10, *LAVE* improves *syntactic@k* from 91.3% to 98.8%, whereas FS-CD shows only a marginal increase from 73.8% to 74.2%. A similar trend is even more evident on `JAVA-Bench` with `DiffuC-7B`: as k increases from 1 to 10, FS-CD shows no improvement at all, whereas *LAVE* improves monotonically throughout. We attribute this behavior to the fact that our approach preserves the non-autoregressive generation mechanism of dLLMs, which provides greater flexibility and diversity during inference.

We further conduct a case study to analyze why *LAVE* outperforms existing approaches. Figure 4 presents a representative example in which only *LAVE* produces a syntactically valid output. In this example, the inference process of IG-CD stalls with a single remaining `[MASK]` token. Theoretically, completing the output at this point requires multiple tokens (such as a variable name, a closing parenthesis, and a semicolon). However, since only one masked position remains, it becomes impossible for the model to generate a grammatically correct completion. We attribute this failure to the unreliable constraints imposed by IG-CD, which allow the model to enter a state where valid continuation is no longer possible. In contrast, FS-CD suffers from severe repetition. As shown in Figure 4, the model repeatedly generates the same `if` statement until the output reaches the maximum generation length, resulting in a forced termination. We attribute this issue to FS-CD's strict enforcement of left-to-right generation, which disrupts the intrinsic non-autoregressive generation mechanism of dLLMs and degrades the quality of the predicted token distributions.

Since dLLMs exhibit randomness during inference, we additionally conduct statistical tests on the results of RQ1. Specifically, we repeat each experiment multiple times and examine the variance

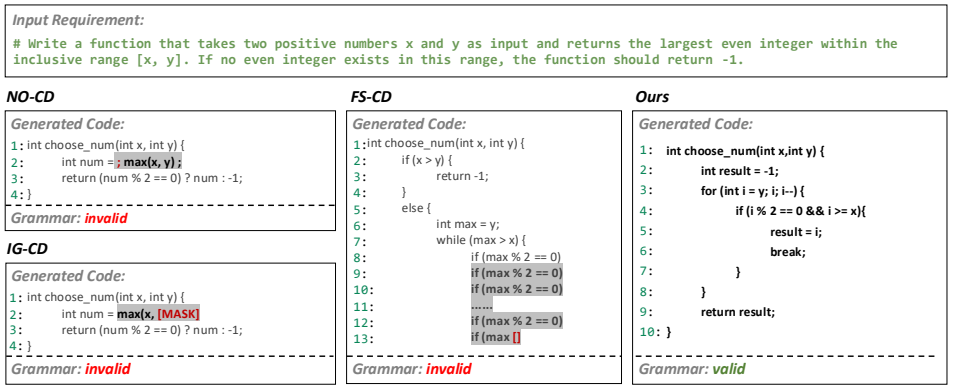


Fig. 4. Outputs generated by NO-CD, FS-CD, IG-CD, and *LAVE* on `Dream-7B` for a problem from `CPP-Bench`.

of all reported metrics. The results show that the maximum variance is only 1.52%, indicating that the randomness has a negligible impact on our conclusions.

Answer to RQ1: *LAVE* effectively enforces CFG constraints on generated outputs and outperforms all baselines across most benchmarks and models. Compared to existing approaches, *LAVE* ensures the validity of every newly generated token throughout the inference process.

5.2 RQ2: How Well Does *LAVE* Improve the Functional Correctness of dLLM Outputs?

Motivation. Although the primary objective of CFG-constrained decoding is to guarantee the syntactic correctness of generated outputs, we are also interested in whether *LAVE* can improve the functional correctness of generated outputs. In this RQ, we evaluate the impact of *LAVE* on the functional correctness of dLLM outputs.

Setting. We apply the baselines and *LAVE* to the four dLLMs described in Section 4.5. We then measure the performance of different approaches on `JSON-Bench`, `CPP-Bench`, `JAVA-Bench`, and `GO-Bench`. As discussed in Section 5.1, IG-CD is difficult to evaluate on `GO-Bench` and `JAVA-Bench`, and is therefore excluded from these two benchmarks. We use *functional@k* as the evaluation metric with $k \in \{1, 5, 10\}$, as defined in Section 4.4. We exclude `SMILES-Bench` because all models yielded a 0% functional correctness rate on this benchmark, making meaningful comparison infeasible.

Results. The *functional@k* of different approaches is reported in Table 2.

***LAVE* consistently improves the functional correctness of dLLMs across four models and four benchmarks.** Compared to the unconstrained baseline, *LAVE* achieves higher *functional@k* in all settings. In contrast, other constrained decoding baselines sometimes degrade model performance. For example, on `CPP-Bench` with `LLaDA-8B`, FS-CD reduces *functional@1* by 0.6%, whereas *LAVE* increases it by 3.2%. A similar pattern can also be observed on `JAVA-Bench`, where, in terms of *functional@10*, FS-CD decreases the score by 6.1%, while *LAVE* improves it by 3.1%.

***LAVE* outperforms other CFG-constrained decoding approaches in improving functional correctness.** The gains achieved by *LAVE* are larger than those of the baseline methods for most settings. For example, on `Dream-7B` with `CPP-Bench`, *LAVE* improves *functional@1*, *functional@5*, and *functional@10* from 25.6%, 26.8%, and 28.1% under the unconstrained setting to 33.5%, 39.5%, and 41.4%, respectively. In contrast, the best baseline reaches only 29.1%, 30.8%, and 31.1%.

***LAVE* yields larger functional correctness gains on tasks with limited reasoning requirements.** `JSON-Bench` focuses on structured information extraction and typically does not require complex logical reasoning, whereas `CPP-Bench` involves code generation and is more dependent

Table 2. *functional@k* (%) on JSON-Bench, CPP-Bench, JAVA-Bench, and GO-Bench, where $k \in \{1, 5, 10\}$. The **best result** for each model, benchmark, and k is highlighted in bold. Absolute differences (in percentage points) are computed with respect to NO-CD.

Method	JSON-Bench			CPP-Bench			JAVA-Bench			GO-Bench		
	k=1	k=5	k=10	k=1	k=5	k=10	k=1	k=5	k=10	k=1	k=5	k=10
LLaDA-8B												
NO-CD	46.9$^{+0.0}$	49.2$^{+0.0}$	49.6$^{+0.0}$	16.3$^{+0.0}$	23.1$^{+0.0}$	24.9$^{+0.0}$	14.0$^{+0.0}$	18.3$^{+0.0}$	21.3$^{+0.0}$	12.8$^{+0.0}$	18.3$^{+0.0}$	19.5$^{+0.0}$
FS-CD	52.9$^{+6.0}$	53.3$^{+4.1}$	53.5$^{+3.9}$	15.7$^{-0.6}$	16.0$^{-7.1}$	16.2$^{-8.7}$	15.1$^{+1.1}$	15.2$^{-3.1}$	15.2$^{-6.1}$	11.6$^{-1.2}$	13.0$^{-5.3}$	13.4$^{-7.9}$
IG-CD	52.1$^{+5.2}$	53.3$^{+4.1}$	53.7$^{+4.1}$	17.8$^{+1.5}$	23.7$^{+0.6}$	25.6$^{+0.7}$	–	–	–	–	–	–
LAVE	**55.0**$^{+8.1}$	**56.3**$^{+7.1}$	**56.4**$^{+6.8}$	**19.5**$^{+3.2}$	**25.9**$^{+2.8}$	**26.7**$^{+1.8}$	**18.3**$^{+4.3}$	**22.0**$^{+3.7}$	**24.4**$^{+3.1}$	**17.7**$^{+4.9}$	**21.3**$^{+3.0}$	**22.6**$^{+3.1}$
LLaDA-1.5												
NO-CD	45.5$^{+0.0}$	46.5$^{+0.0}$	46.7$^{+0.0}$	17.2$^{+0.0}$	22.0$^{+0.0}$	25.9$^{+0.0}$	18.3$^{+0.0}$	21.3$^{+0.0}$	22.6$^{+0.0}$	15.2$^{+0.0}$	16.5$^{+0.0}$	17.7$^{+0.0}$
FS-CD	53.1$^{+7.6}$	53.6$^{+7.1}$	53.6$^{+6.9}$	15.9$^{-1.3}$	16.5$^{-5.5}$	16.5$^{-9.4}$	15.2$^{-3.1}$	15.2$^{-6.1}$	15.2$^{-7.4}$	15.7$^{+0.5}$	15.9$^{-0.6}$	15.9$^{-1.8}$
IG-CD	50.6$^{+5.1}$	52.2$^{+5.8}$	52.5$^{+5.8}$	18.3$^{+1.1}$	23.1$^{+1.1}$	27.1$^{+1.2}$	–	–	–	–	–	–
LAVE	**54.1**$^{+8.6}$	**54.7**$^{+8.2}$	**54.7**$^{+8.0}$	**19.1**$^{+1.9}$	**24.9**$^{+2.9}$	**28.8**$^{+2.9}$	**20.7**$^{+2.4}$	**23.2**$^{+1.9}$	**24.4**$^{+1.8}$	**18.3**$^{+3.1}$	**19.1**$^{+2.6}$	**19.5**$^{+1.8}$
Dream-7B												
NO-CD	49.8$^{+0.0}$	50.7$^{+0.0}$	50.7$^{+0.0}$	25.6$^{+0.0}$	26.8$^{+0.0}$	28.1$^{+0.0}$	24.4$^{+0.0}$	25.6$^{+0.0}$	25.6$^{+0.0}$	24.4$^{+0.0}$	26.8$^{+0.0}$	26.8$^{+0.0}$
FS-CD	52.9$^{+3.1}$	53.2$^{+2.5}$	53.2$^{+2.5}$	26.5$^{+0.9}$	27.1$^{+0.3}$	27.5$^{-0.6}$	25.0$^{+0.6}$	25.0$^{-0.6}$	25.0$^{-0.6}$	27.9$^{+3.5}$	28.0$^{+1.2}$	28.0$^{+1.2}$
IG-CD	51.5$^{+1.7}$	53.5$^{+2.8}$	53.9$^{+3.2}$	29.1$^{+3.5}$	30.8$^{+4.0}$	31.1$^{+3.0}$	–	–	–	–	–	–
LAVE	**54.4**$^{+4.6}$	**55.1**$^{+4.4}$	**55.5**$^{+4.8}$	**33.5**$^{+7.9}$	**39.5**$^{+12.7}$	**41.4**$^{+13.3}$	**28.1**$^{+3.7}$	**29.3**$^{+3.7}$	**30.5**$^{+4.9}$	**30.8**$^{+6.4}$	**32.3**$^{+5.5}$	**33.1**$^{+6.3}$
DiffuC-7B												
NO-CD	52.0$^{+0.0}$	53.3$^{+0.0}$	53.7$^{+0.0}$	37.2$^{+0.0}$	37.8$^{+0.0}$	37.8$^{+0.0}$	29.3$^{+0.0}$	30.5$^{+0.0}$	30.5$^{+0.0}$	22.0$^{+0.0}$	22.6$^{+0.0}$	23.3$^{+0.0}$
FS-CD	53.7$^{+1.7}$	54.1$^{+0.8}$	54.1$^{+0.4}$	35.2$^{-2.0}$	35.8$^{-2.0}$	35.9$^{-1.9}$	29.3$^{+0.0}$	29.7$^{-0.8}$	29.9$^{-0.6}$	23.2$^{+1.2}$	23.2$^{+0.6}$	23.2$^{-0.1}$
IG-CD	53.0$^{+1.0}$	55.3$^{+2.0}$	55.3$^{+1.6}$	**38.1**$^{+0.9}$	38.3$^{+0.5}$	**38.5**$^{+0.7}$	–	–	–	–	–	–
LAVE	**55.2**$^{+3.2}$	**56.6**$^{+3.3}$	**56.7**$^{+3.0}$	**38.1**$^{+0.9}$	**38.5**$^{+0.7}$	**38.5**$^{+0.7}$	**32.2**$^{+2.9}$	**32.9**$^{+2.4}$	**32.9**$^{+2.4}$	**24.4**$^{+2.2}$	**25.8**$^{+3.2}$	**26.2**$^{+2.9}$

on reasoning ability. We observe that, compared to NO-CD, *LAVE* achieves larger improvements in *functional@k* on JSON-Bench than on CPP-Bench. For example, with LLaDA-8B, *LAVE* improves functional correctness by 8.1% on JSON-Bench, whereas the improvement is 3.2% on CPP-Bench. This behavior is expected because, in tasks with limited reasoning requirements, functional correctness depends more strongly on syntactic validity, allowing the syntactic improvements provided by *LAVE* to translate more effectively into functional gains.

Similar to RQ1, we repeat each experiment multiple times and examine the variance of all reported metrics. The results show that the maximum standard deviation is only 0.98%.

Answer to RQ2: Compared to baselines, *LAVE* improves the functional correctness of dLLMs more consistently and significantly. Furthermore, this improvement is particularly evident in tasks with lower logical reasoning requirements.

5.3 RQ3: What Runtime Overhead Does *LAVE* Introduce?

Motivation. Beyond the quality of generated outputs, runtime efficiency is also a critical factor for the practical adoption of constrained decoding approaches. If a constrained decoding method introduces excessive inference overhead, its practical utility would be substantially limited [3]. Therefore, in this RQ, we examine the runtime overhead introduced by *LAVE*.

Setting. We apply the baselines and *LAVE* to the four models described in Section 4.5 and evaluate them on the five benchmarks introduced in Section 4.1. We use average inference time as the metric to measure the runtime overhead of different approaches. As discussed in Section 5.1, IG-CD

Table 3. Average inference time (s) on JSON-Bench, CPP-Bench, JAVA-Bench, GO-Bench, and SMILES-Bench. The best result for each model and benchmark is highlighted in bold. The last row reports the relative inference time normalized by NO-CD.

Model	JSON-Bench				CPP-Bench				JAVA-Bench			GO-Bench			SMILES-Bench			
	NO-CD	FS-CD	IG-CD	*LAVE*	NO-CD	FS-CD	IG-CD	*LAVE*	NO-CD	FS-CD	*LAVE*	NO-CD	FS-CD	*LAVE*	NO-CD	FS-CD	IG-CD	*LAVE*
LLaDA-8B	9.48	13.29	10.34	**8.22**	**8.55**	16.68	9.66	9.30	**8.03**	8.74	8.56	**8.09**	10.19	8.32	8.49	7.68	8.76	**3.02**
LLaDA-1.5	**9.71**	14.74	10.82	9.91	**8.56**	11.88	12.17	12.02	**8.16**	10.29	9.22	**8.23**	8.28	9.08	8.77	**2.16**	8.21	2.97
Dream-7B	8.62	15.74	**8.25**	9.73	7.46	**7.17**	7.95	7.26	7.37	**6.30**	6.41	**7.05**	10.01	7.67	7.07	8.99	19.64	**2.24**
DiffuC-7B	**8.15**	15.39	9.78	9.22	**7.45**	14.78	8.47	12.27	7.38	**6.81**	7.52	**7.03**	7.11	7.40	6.86	11.15	6.74	**4.41**
Average	**8.99**	14.79	9.80	9.27	**8.01**	12.63	9.56	10.21	**7.74**	8.04	7.93	**7.60**	8.90	8.12	7.80	7.50	10.84	**3.16**
Ratio	×1.00	×1.65	×1.09	×1.03	×1.00	×1.58	×1.19	×1.27	×1.00	×1.04	×1.02	×1.00	×1.17	×1.07	×1.00	×0.96	×1.39	×0.41

Table 4. Ablation study of *LAVE*. Verification and Recovery denote Lookahead-Based Verification and Cache-Enhanced Recovery, respectively. Syn and Func represent *syntactic@1* and *functional@1*, respectively, while Time denotes the average inference time.

Model	Verification	Recovery	JSON-Bench			CPP-Bench		
			Syn. (%)	Func. (%)	Time (s)	Syn. (%)	Func. (%)	Time (s)
LLaDA-8B	✗	✗	85.4 (+0.0)	46.9 (+0.0)	9.48	74.2 (+0.0)	16.3 (+0.0)	8.55
	✓	✗	95.8 (+10.4)	53.7 (+6.8)	8.09	86.6 (+12.4)	18.9 (+2.6)	8.71
	✓	✓	99.5 (+14.1)	55.0 (+8.1)	8.22	90.9 (+16.7)	19.5 (+3.2)	9.30
Dream-7B	✗	✗	86.8 (+0.0)	49.8 (+0.0)	8.62	76.2 (+0.0)	25.6 (+0.0)	7.46
	✓	✗	94.8 (+8.0)	51.8 (+2.0)	9.17	92.7 (+16.5)	32.3 (+6.7)	7.69
	✓	✓	99.0 (+12.2)	54.4 (+4.6)	9.73	96.0 (+19.8)	33.5 (+7.9)	7.26

is difficult to evaluate on GO-Bench and JAVA-Bench, and is therefore excluded from these two benchmarks.

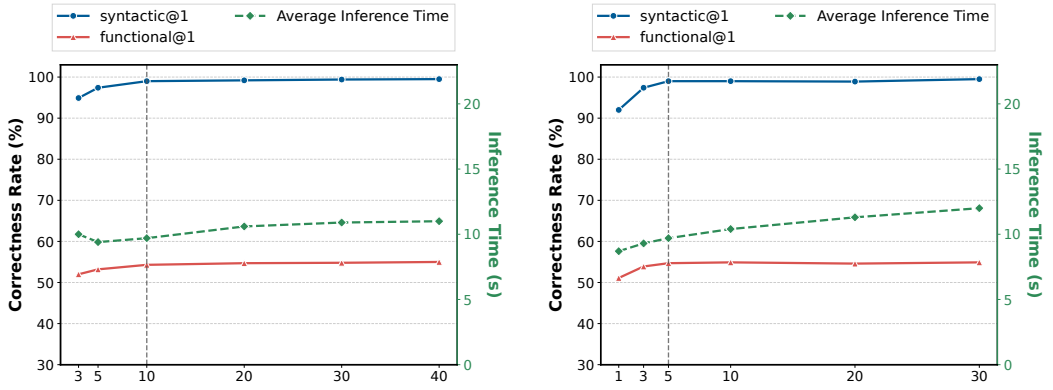
Results. The average inference times of different approaches are reported in Table 3. Lower values indicate better efficiency.

***LAVE* introduces negligible runtime overhead across most benchmarks and models.** For example, on LLaDA-1.5 evaluated on JSON-Bench, applying *LAVE* increases the average inference time from 9.71 seconds to 9.91 seconds, corresponding to an increase of only 2%. For LLaDA-8B on CPP-Bench, where the grammar of C++ is considerably more complex than that of JSON or SMILES, the inference time increases modestly from 8.55 seconds to 9.30 seconds, corresponding to a 9% increase. Interestingly, we observe that in several scenarios, the average inference time of *LAVE* is even lower than that of the unconstrained baseline. For example, with DiffuC-7B on SMILES-Bench, the average inference time for *LAVE* is 4.41 seconds, compared to 6.86 seconds for the unconstrained setting. We attribute this phenomenon to the fact that, without constraints, the model often generates syntactically invalid outputs (such as extraneous natural language explanations), resulting in more generated tokens and consequently prolonging the inference process.

Answer to RQ3: *LAVE* introduces negligible runtime overhead in most settings, allowing it to improve the reliability of structured generation in dLLMs with minimal additional cost.

5.4 RQ4: How Does Each Component of *LAVE* Contribute to the Overall Performance?

Motivation. *LAVE* consists of two components, namely Lookahead-Based Verification and Cache-Enhanced Recovery. In this RQ, we evaluate the contribution of each component through ablation experiments.



(a) Lookahead size N (b) Proposal budget τ

Fig. 5. The performance of *LAVE* with different values of the lookahead size N and proposal budget τ. We use the gray dashed line to represent the employed hyperparameters.

Setting. Due to the high computational cost, we select two representative models and two representative benchmarks for ablation experiments. Specifically, we conduct experiments on `LLaDA-8B` and `Dream-7B` using `JSON-Bench` and `CPP-Bench`. All experiments are evaluated using the three metrics defined in Section 4.4. Since Cache-Enhanced Recovery depends on Lookahead-Based Verification, it is not possible to remove Lookahead-Based Verification while retaining Cache-Enhanced Recovery. Therefore, we consider three experimental settings: ❶ retaining both Lookahead-Based Verification and Cache-Enhanced Recovery, corresponding to *LAVE*; ❷ retaining Lookahead-Based Verification while removing Cache-Enhanced Recovery, where the model switches to unconstrained generation once the triggering condition for Cache-Enhanced Recovery is met; ❸ removing both Lookahead-Based Verification and Cache-Enhanced Recovery, which corresponds to NO-CD.

Results. The results of the ablation experiments are reported in Table 4.

Lookahead-Based Verification alone significantly improves both syntactic and functional correctness. For example, on `LLaDA-8B` evaluated on `JSON-Bench`, applying Lookahead-Based Verification increases the *syntactic@1* by 10.4% and the *functional@1* by 6.8%. On `Dream-7B` evaluated on `CPP-Bench`, applying Lookahead-Based Verification increases the *syntactic@1* by 16.5% and the *functional@1* by 6.7%.

Cache-Enhanced Recovery further improves the performance of *LAVE* with negligible additional runtime overhead. Across all evaluated settings, Cache-Enhanced Recovery consistently improves both syntactic and functional correctness compared to using Lookahead-Based Verification alone, while having a negligible impact on average inference time. For example, on `Dream-7B` evaluated on `JSON-Bench`, it increases the *syntactic@1* to 99.0% and the *functional@1* to 54.4%, with an added runtime cost of less than 0.6 seconds.

Answer to RQ4: Both Lookahead-Based Verification and Cache-Enhanced Recovery contribute effectively to the performance of *LAVE* by improving the syntactic and functional correctness of dLLM outputs.

5.5 RQ5: How Do the Hyperparameters Affect the Performance of *LAVE*?

Motivation. *LAVE* involves several hyperparameters that may affect its performance. In this RQ, we conduct a sensitivity analysis to examine their impact. Specifically, we consider two categories of hyperparameters. ① The first category consists of hyperparameters introduced by *LAVE*, including

Table 5. Performance of *LAVE* with varying values of the hyperparameter T. The gray background indicates the hyperparameter setting used in our experiments.

Method	*syntactic@1* (%)					*functional@1* (%)					Average Inference Time (s)				
	T=16	T=32	T=64	T=128	T=256	T=16	T=32	T=64	T=128	T=256	T=16	T=32	T=64	T=128	T=256
NO-CD	7.8	21.0	63.6	86.8	97.4	1.1	7.4	40.4	49.8	54.8	1.98	2.92	4.56	8.62	15.88
LAVE	76.3	91.7	97.2	99.0	100.0	20.4	32.2	46.6	54.4	54.8	3.21	3.76	5.48	9.73	16.02

the lookahead size N, defined in Section 3.2, and the attempt budget τ, defined in Section 3.3. ② The second category includes key hyperparameters inherent to dLLMs, for which we select the number of denoising steps T.

Setting. Due to the high computational cost, we conduct experiments on `Dream-7B` using `JSON-Bench`. All experiments are evaluated using the three metrics defined in Section 4.4. For the lookahead size N, we consider values in $\{3, 5, 10, 20, 30, 40\}$. For the attempt budget τ, we consider values in $\{1, 3, 5, 10, 20, 30\}$. For the number of denoising steps T, we consider values in $\{16, 32, 64, 128, 256\}$.

Results.

The results for the hyperparameters introduced by LAVE, namely the lookahead size N and the attempt budget τ, are shown in Figure 5. ❶ **Effect of the lookahead size N.** We observe that increasing N generally leads to improvements in both *syntactic@1* and *functional@1*. This trend can be attributed to the fact that a larger N increases the likelihood of identifying at least one valid *complete prefix* during Lookahead-Based Verification. Interestingly, the average inference time first decreases and then increases as N grows. This behavior reflects two competing effects. On the one hand, a larger N increases the number of *complete prefixes* evaluated during each validation step, which increases inference time. On the other hand, a larger N increases the probability that a valid proposed token is accepted, thereby reducing the number of re-proposal attempts and shortening the overall inference time. ❷ **Effect of the proposal budget τ.** As τ increases, all three metrics exhibit an upward trend. A larger attempt budget allows the model more opportunities to propose high-confidence tokens, which improves both syntactic and functional correctness. At the same time, increasing τ leads to more re-proposal, which increases inference time. ❸ **Overall robustness to hyperparameter variations.** Overall, *LAVE* demonstrates strong robustness to adjustments in the newly introduced hyperparameters. Although varying lookahead size N and τ leads to observable performance trends, the magnitude of these changes remains limited. For example, when N increases from 3 to 40, the average inference time remains stable at around 10 seconds. Similarly, when τ increases from 3 to 30, the *syntactic@1* consistently remains above 97%.

The results for the hyperparameters inherent to dLLMs, namely the denoising steps T, are shown in Table 5. ❶ ***LAVE* consistently improves syntactic and functional correctness across different values of T.** For instance, when T is set to 32, which in our setting corresponds to generating at least eight tokens per forward pass, *LAVE* increases the *syntactic@1* from 21.0% to 91.7% and the *functional@1* from 7.4% to 32.2%. Although the runtime overhead increases slightly when the number of denoising steps is small (*e.g.,* rising from 2.92s to 3.76s at $T = 32$), we consider this cost acceptable given the substantial gains in correctness. ❷ **The syntactic and functional correctness of dLLMs with *LAVE* increase as T increases.** For instance, as T increases from 16 to 128, the *syntactic@1* improves from 76.3% to 99.0%, and the *functional@1* rises from 20.4% to 54.4%. We attribute this to the characteristics of dLLMs, where a higher number of denoising steps leads to better effectiveness but lower efficiency. The unconstrained baseline exhibits a similar pattern, further supporting this explanation.

Table 6. Comparison with AR LLMs. The results are reported by *syntactic@k* and *functional@k* under $k \in \{1, 5, 10\}$ on JSON-Bench and CPP-Bench.

Model	Method	JSON-Bench						CPP-Bench					
		syntactic@k			*functional@k*			*syntactic@k*			*functional@k*		
		$k=1$	$k=5$	$k=10$	$k=1$	$k=5$	$k=10$	$k=1$	$k=5$	$k=10$	$k=1$	$k=5$	$k=10$
LLaDA-8B	w/o CD	91.2	91.2	91.2	50.0	50.7	51.1	98.2	98.8	98.8	26.2	29.3	31.7
	w/ CD	99.6	99.9	**100.0**	54.0	54.8	55.2	**99.4**	**100.0**	**100.0**	26.8	30.5	32.3
Dream-7B	w/o CD	97.4	97.6	97.6	54.8	55.2	55.2	97.0	97.6	98.2	37.8	39.0	39.0
	w/ CD	**100.0**	**100.0**	**100.0**	54.8	56.3	57.0	98.8	99.4	99.4	39.0	39.9	40.2
LLaMA-8B	w/o CD	92.3	93.4	94.9	55.2	57.4	57.7	93.3	96.3	97.0	37.8	44.5	47.6
	w/ CD	99.6	99.6	99.6	56.3	57.4	57.7	94.2	96.6	97.0	38.4	45.1	48.2
Qwen-7B	w/o CD	98.2	98.9	98.9	57.4	**59.6**	**60.7**	93.3	97.6	97.6	**60.4**	68.9	69.5
	w/ CD	**100.0**	**100.0**	**100.0**	**57.9**	**59.6**	**60.7**	98.6	98.8	98.8	**60.4**	**69.5**	**70.7**

Answer to RQ5: *LAVE* exhibits strong robustness to variations in key hyperparameters and consistently improves the quality of generated outputs across different settings.

6 Discussion

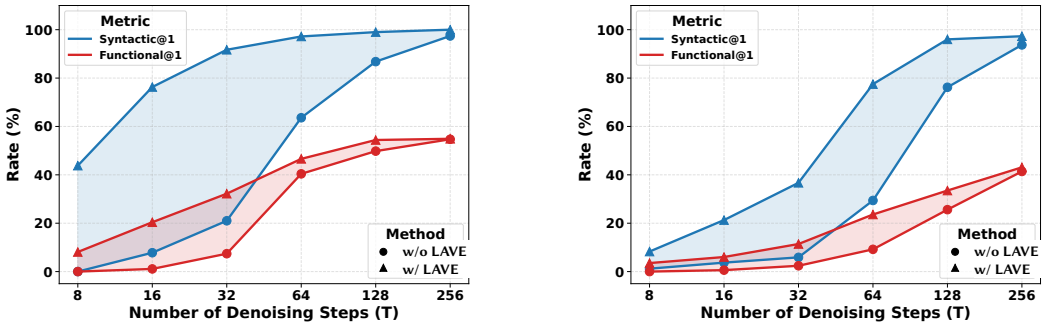
6.1 Understanding the Remaining Syntax Errors

Although our approach guarantees reliable grammar constraints by ensuring that no newly generated token renders the intermediate output impossible to complete into a valid sentence, occasional syntax errors still occur (Section 5.1). We identify the root cause of these remaining errors as the model's tendency to generate repetitive patterns, a phenomenon widely observed in previous studies [11, 16, 22]. In such cases, the dLLM repeatedly generates specific content (*e.g.,* whitespace or newline characters) until it reaches the maximum generation length. To confirm this, we examined all invalid outputs from our experiments and found that the number of generated tokens exactly matched the maximum generation length in every case. Fortunately, the majority of approaches [9, 22, 50] designed to resolve such repetition problems are orthogonal to our approach; therefore, they can be effectively combined with *LAVE* to further improve the reliability of dLLM generation.

6.2 Comparison with AR LLMs

We further compare dLLMs with AR LLMs of similar scale on JSON-Bench and CPP-Bench. Specifically, we evaluate two representative dLLMs, LLaDA-8B and Dream-7B, and compare them with Llama-3-8B-Instruct and Qwen2.5-7B-Instruct, denoted as LLaMA-8B and Qwen-7B. For dLLMs, we compare unconstrained decoding with *LAVE*; for AR LLMs, we compare unconstrained decoding with LLGuidance [28]. For fairness, similar to AR LLMs, dLLMs are configured to decode one token per step. The results are shown in Table 6.

❶ ***LAVE* enables dLLMs to achieve strong syntactic correctness.** For example, on CPP-Bench, *LAVE* improves the *syntactic@5* of LLaDA-8B to 100.0%, outperforming both constrained AR LLMs. On JSON-Bench, both dLLMs also achieve nearly perfect syntactic correctness with *LAVE*, confirming its effectiveness in enforcing CFG constraints. ❷ **Current dLLMs still lag behind AR LLMs in functional correctness.** Despite the syntactic gains, dLLMs remain generally weaker than strong AR LLMs in functional correctness. This mainly reflects the capability gap between current dLLMs and mature AR LLMs, suggesting that improving dLLM foundational capabilities is complementary to our work. Moreover, although Dream-7B shows stronger functional correctness



(a) JSON-Bench (b) CPP-Bench

Fig. 6. Performance of *LAVE* under different numbers of denoising steps T on JSON-Bench and CPP-Bench using Dream-7B.

than LLaDA-8B, *LAVE* still consistently improves its correctness, further supporting that our approach is complementary to improvements in the foundational capabilities of dLLMs.

6.3 Conditions for Efficient Lookahead

The efficiency of *LAVE* relies on the empirical observation that only a small number of complete-prefix samples is usually sufficient for Lookahead-Based Verification (Section 3.2.2). We acknowledge that this is not a formal guarantee: when *the model is too weak, the grammar is too complex, or the problem is too difficult*, a larger lookahead size N may be needed. Nevertheless, our experimental results show that this condition holds well across four popular dLLMs and five representative benchmarks, where *LAVE* consistently improves correctness with only modest runtime overhead. Importantly, this condition is used to justify the practicality of *LAVE* rather than the reliability of its design. The core contribution of *LAVE* is the reliability-oriented verification criterion, while the empirical small-N observation explains why this criterion is affordable in practice.

6.4 Understanding *LAVE* from a Correctness–Efficiency Perspective

Although *LAVE* is primarily designed to improve the syntactic correctness of dLLM outputs, its effect can also be understood from a correctness–efficiency perspective. The efficiency advantage of dLLMs mainly comes from parallel decoding: with fewer denoising steps, the model can generate more tokens per step. However, this efficiency often comes at the cost of lower correctness. Therefore, dLLM decoding naturally involves a trade-off between correctness and efficiency.

LAVE improves this trade-off by increasing correctness under the same denoising budget. To examine this effect, we evaluate *LAVE* with Dream-7B on JSON-Bench and CPP-Bench under different numbers of denoising steps, as shown in Figure 6. Compared with unconstrained decoding, *LAVE* achieves higher correctness across different denoising budgets. Therefore, *LAVE* may not be viewed only as a method that improves syntactic correctness. It also provides a better correctness–efficiency trade-off, making dLLMs more practical in latency-sensitive scenarios.

6.5 Threats to Validity

We identify three primary threats to the validity of our study:

❶ **Generalizability of the Experimental Results.** A potential concern is whether our findings hold across diverse tasks and models. To mitigate this threat, we carefully selected a broad range of benchmarks and models for our evaluation. Specifically, we adopted five representative benchmarks covering code generation, information extraction, and scientific applications. These domains reflect

typical real-world scenarios where structured outputs are essential, aligning with setups in prior research [29]. Furthermore, we evaluated our approach on four widely used dLLMs, ensuring that our conclusions are not artifacts of a specific model architecture or training data distribution.

❷ **Reliability-Oriented Constraint Design.** Due to the non-autoregressive generation mechanism of dLLMs, it is inherently difficult to guarantee that all accepted tokens are valid while all rejected tokens are invalid. As a result, our approach may occasionally reject proposed tokens that could still lead to valid completions. Nevertheless, we argue that the primary goal of constrained decoding is to improve the reliability of generated outputs. From this perspective, ensuring that all accepted tokens preserve grammatical validity is more important than accepting every potentially valid token. Empirically, Section 3.2.2 suggests that a small lookahead budget is usually sufficient to uncover an extendable candidate when one exists, reducing the likelihood of rejecting valid tokens. The results in Section 5.1 and Section 5.2 further show that our method consistently improves syntactic correctness and also increases functional correctness, indicating that the practical impact of rejecting some valid tokens is negligible.

❸ **Impact of Hyperparameters.** Our approach introduces two hyperparameters, which could potentially limit utility if the approach requires extensive tuning. To address this, we initially selected reasonable values based on intuition and observed consistent performance improvements across multiple benchmarks and models. To rigorously examine the sensitivity of these parameters, we conducted a sensitivity analysis by varying their values over a wide range. The empirical results indicate that our approach demonstrates strong robustness to variations in hyperparameter settings, maintaining high performance without the need for precise tuning.

7 Conclusion

In this paper, we present *LAVE*, a new constrained decoding approach tailored to emerging diffusion LLMs. *LAVE* exploits the unique ability of these models to predict token distributions for all positions in parallel during a single forward pass. Our approach addresses the limitations of existing constrained decoding methods by enforcing reliable grammar constraints that preserve the potential for intermediate outputs to be extended into valid sentences. Experimental results on code generation, JSON generation, and chemical expression generation demonstrate that *LAVE* can effectively improve the reliability of dLLMs in producing CFG-compliant outputs.

Data Availability

The source code is provided at: https://github.com/THU-Agent/LAVE.

Acknowledgments

This work was supported by the Tsinghua University–Keystone Electrical (Zhejiang) Co., Ltd. Joint Research Center for Embodied Multimodal Artificial Intelligence (JCEMAI) and the Beijing Natural Science Foundation under Grant No. 4264107.

This work was also supported by the Beijing Natural Science Foundation under Grant No. QY24136, the National Natural Science Foundation of China under Grant No. 62192733, 62192730, 62192731, the National Key R&D Program under Grant No. 2023YFB4503801, the Beijing Major Science and Technology Project under Contract no. Z251100008425005.

References

[1] Alfred V. Aho and Stephen C. Johnson. 1974. LR parsing. *ACM Computing Surveys (CSUR)* 6, 2 (1974), 99–124.

[2] Marianne Arriola, Aaron Gokaslan, Justin T Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. 2025. Block diffusion: Interpolating between autoregressive and diffusion language models. *arXiv preprint arXiv:2503.09573* (2025).

[3] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. 2024. Guiding llms the right way: Fast, non-invasive constrained generation. *arXiv preprint arXiv:2403.06988* (2024).

[4] Junyi Chen, Shihao Bai, Zaijun Wang, Siyu Wu, Chuheng Du, Hailong Yang, Ruihao Gong, Shengzhong Liu, Fan Wu, and Guihai Chen. 2025. Pre^3: Enabling Deterministic Pushdown Automata for Faster Structured LLM Generation. *arXiv preprint arXiv:2506.03887* (2025).

[5] Mark Chen. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[6] Zicong Cheng, Guo-Wei Yang, Jia Li, Zhijie Deng, Meng-Hao Guo, and Shi-Min Hu. 2025. DEER: Draft with Diffusion, Verify with Autoregressive Models. *arXiv preprint arXiv:2512.15176* (2025).

[7] Google DeepMind. 2025. Gemini Diffusion. https://deepmind.google/models/gemini-diffusion/

[8] Franklin L DeRemer. 1971. Simple LR (k) grammars. *Commun. ACM* 14, 7 (1971), 453–460.

[9] Yihong Dong, Yuchen Liu, Xue Jiang, Bin Gu, Zhi Jin, and Ge Li. 2025. Rethinking repetition problems of llms in code generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 965–985.

[10] Jay Earley. 1970. An efficient context-free parsing algorithm. *Commun. ACM* 13, 2 (1970), 94–102.

[11] Zihao Fu, Wai Lam, Anthony Man-Cho So, and Bei Shi. 2021. A theoretical analysis of the repetition problem in text generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35. 12848–12856.

[12] Yifeng Gao, Ziang Ji, Yuxuan Wang, Biqing Qi, Hanlin Xu, and Linfeng Zhang. 2025. Self Speculative Decoding for Diffusion Large Language Models. *arXiv preprint arXiv:2510.04147* (2025).

[13] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. 2023. Grammar-constrained decoding for structured NLP tasks without finetuning. *arXiv preprint arXiv:2305.13971* (2023).

[14] Shansan Gong, Shivam Agarwal, Yizhe Zhang, Jiacheng Ye, Lin Zheng, Mukai Li, Chenxin An, Peilin Zhao, Wei Bi, Jiawei Han, et al. 2024. Scaling diffusion language models via adaptation from autoregressive models. *arXiv preprint arXiv:2410.17891* (2024).

[15] Shansan Gong, Ruixiang Zhang, Huangjie Zheng, Jiatao Gu, Navdeep Jaitly, Lingpeng Kong, and Yizhe Zhang. 2025. DiffuCoder: Understanding and Improving Masked Diffusion Models for Code Generation. *arXiv preprint arXiv:2506.20639* (2025).

[16] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. 2019. The curious case of neural text degeneration. *arXiv preprint arXiv:1904.09751* (2019).

[17] Zhanqiu Hu, Jian Meng, Yash Akhauri, Mohamed S Abdelfattah, Jae-sun Seo, Zhiru Zhang, and Udit Gupta. 2025. Accelerating diffusion language model inference via efficient kv caching and guided diffusion. *arXiv preprint arXiv:2505.21467* (2025).

[18] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. 2024. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186* (2024).

[19] Xue Jiang, Yihong Dong, Yongding Tao, Huanyu Liu, Zhi Jin, Wenpin Jiao, and Ge Li. 2024. Rocode: Integrating backtracking mechanism and program analysis in large language models for code generation. *arXiv preprint arXiv:2411.07112* (2024).

[20] Samar Khanna, Siddhant Kharbanda, Shufan Li, Harshit Varma, Eric Wang, Sawyer Birnbaum, Ziyang Luo, Yanis Miraoui, Akash Palrecha, Stefano Ermon, Aditya Grover, and Volodymyr Kuleshov. 2025. Mercury: Ultra-Fast Language Models Based on Diffusion. *CoRR* abs/2506.17298 (2025). arXiv:2506.17298 doi:10.48550/ARXIV.2506.17298

[21] Chengze Li, Yitong Zhang, Jia Li, Liyi Cai, Ge Li, et al. 2025. Beyond Autoregression: An Empirical Study of Diffusion Large Language Models for Code Generation. *arXiv preprint arXiv:2509.11252* (2025).

[22] Huayang Li, Tian Lan, Zihao Fu, Deng Cai, Lemao Liu, Nigel Collier, Taro Watanabe, and Yixuan Su. 2023. Repetition in repetition out: Towards understanding neural text degeneration from the data perspective. *Advances in Neural Information Processing Systems* 36 (2023), 72888–72903.

[23] Tianyi Li, Mingda Chen, Bowei Guo, and Zhiqiang Shen. 2025. A survey on diffusion language models. *arXiv preprint arXiv:2508.10875* (2025).

[24] Zherui Li, Zheng Nie, Zhenhong Zhou, Yufei Guo, Yue Liu, Yitong Zhang, Yu Cheng, Qingsong Wen, Kun Wang, and Jiaheng Zhang. 2025. Diffuguard: How intrinsic safety is lost and found in diffusion large language models. *arXiv preprint arXiv:2509.24296* (2025).

[25] Aiwei Liu, Minghua He, Shaoxun Zeng, Sijun Zhang, Linhao Zhang, Chuhan Wu, Wei Jia, Yuan Liu, Xiao Zhou, and Jie Zhou. 2025. WeDLM: Reconciling Diffusion Language Models with Standard Causal Attention for Fast Inference. *arXiv preprint arXiv:2512.22737* (2025).

[26] João Loula, Benjamin LeBrun, Li Du, Ben Lipkin, Clemente Pasti, Gabriel Grand, Tianyu Liu, Yahya Emara, Marjorie Freedman, Jason Eisner, et al. 2025. Syntactic and semantic control of large language models via sequential monte carlo. *arXiv preprint arXiv:2504.13139* (2025).

[27] Daniel Melcer, Nathan Fulton, Sanjay Krishna Gouda, and Haifeng Qian. 2024. Constrained decoding for code language models via efficient left and right quotienting of context-sensitive grammars. *CoRR* (2024).

[28] Microsoft. 2025. LLGuidance. https://github.com/guidance-ai/llguidance?tab=readme-ov-file
[29] Niels Mündler, Jasper Dekoninck, and Martin Vechev. 2025. Constrained decoding of diffusion llms with context-free grammars. *arXiv preprint arXiv:2508.10111* (2025).
[30] Nishanth Sridhar Nakshatri, Shamik Roy, Rajarshi Das, Suthee Chaidaroon, Leonid Boytsov, and Rashmi Gangadharaiah. 2025. Constrained decoding with speculative lookaheads. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 4681–4700.
[31] Quan Nguyen-Tri, Mukul Ranjan, and Zhiqiang Shen. 2025. Attention is all you need for kv cache in diffusion llms. *arXiv preprint arXiv:2510.14973* (2025).
[32] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. 2025. Large language diffusion models. *arXiv preprint arXiv:2502.09992* (2025).
[33] NousResearch. 2024. json-mode-eval. https://huggingface.co/datasets/NousResearch/json-mode-eval. Hugging Face Datasets.
[34] Kanghee Park, Timothy Zhou, and Loris D'Antoni. 2025. Flexible and efficient grammar-constrained decoding. *arXiv preprint arXiv:2502.05111* (2025).
[35] PaweŁ Parys, Sairam Vaidya, Taylor Berg-Kirkpatrick, and Loris D'Antoni. 2025. Constrained Adaptive Rejection Sampling. *arXiv preprint arXiv:2510.01902* (2025).
[36] Subham Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin Chiu, Alexander Rush, and Volodymyr Kuleshov. 2024. Simple and effective masked diffusion language models. *Advances in Neural Information Processing Systems* 37 (2024), 130136–130184.
[37] Yuxuan Song, Zheng Zhang, Cheng Luo, Pengyang Gao, Fan Xia, Hao Luo, Zheng Li, Yuehang Yang, Hongli Yu, Xingwei Qu, et al. 2025. Seed diffusion: A large-scale diffusion language model with high-speed inference. *arXiv preprint arXiv:2508.02193* (2025).
[38] Xintong Sun, Chi Wei, Minghao Tian, and Shiwen Ni. 2025. Earley-Driven Dynamic Pruning for Efficient Structured Decoding. *arXiv preprint arXiv:2506.01151* (2025).
[39] Tarun Suresh, Debangshu Banerjee, Shubham Ugare, Sasa Misailovic, and Gagandeep Singh. 2025. DINGO: Constrained Inference for Diffusion LLMs. *arXiv preprint arXiv:2505.23061* (2025).
[40] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
[41] Shubham Ugare, Rohan Gumaste, Tarun Suresh, Gagandeep Singh, and Sasa Misailovic. 2024. IterGen: Iterative Semantic-aware Structured LLM Generation with Backtracking. *arXiv preprint arXiv:2410.07295* (2024).
[42] Shubham Ugare, Tarun Suresh, Hangoo Kang, Sasa Misailovic, and Gagandeep Singh. 2024. Improving llm code generation with grammar augmentation. *arXiv preprint arXiv:2403.01632* 19 (2024).
[43] Shubham Ugare, Tarun Suresh, Hangoo Kang, Sasa Misailovic, and Gagandeep Singh. 2024. SynCode: LLM generation with grammar augmentation. *Transactions on Machine Learning Research* (2024).
[44] Tim A Wagner and Susan L Graham. 1998. Efficient and flexible incremental parsing. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 20, 5 (1998), 980–1013.
[45] Xu Wang, Chenkai Xu, Yijie Jin, Jiachun Jin, Hao Zhang, and Zhijie Deng. 2025. Diffusion llms can do faster-than-ar inference via discrete diffusion forcing. *arXiv preprint arXiv:2508.09192* (2025).
[46] Yinjie Wang, Ling Yang, Bowen Li, Ye Tian, Ke Shen, and Mengdi Wang. 2025. Revolutionizing reinforcement learning framework for diffusion large language models. *arXiv preprint arXiv:2509.06949* (2025).
[47] Zhuoer Wang, Leonardo FR Ribeiro, Alexandros Papangelis, Rohan Mukherjee, Tzu-Yen Wang, Xinyan Zhao, Arijit Biswas, James Caverlee, and Angeliki Metallinou. 2024. FANTAstic SEquences and where to find them: Faithful and efficient API call generation through state-tracked constrained decoding and reranking. *arXiv preprint arXiv:2407.13945* (2024).
[48] Brandon T Willard and Rémi Louf. 2023. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702* (2023).
[49] Zhihui Xie, Jiacheng Ye, Lin Zheng, Jiahui Gao, Jingwei Dong, Zirui Wu, Xueliang Zhao, Shansan Gong, Xin Jiang, Zhenguo Li, et al. 2025. Dream-coder 7b: An open diffusion language model for code. *arXiv preprint arXiv:2509.01142* (2025).
[50] Jin Xu, Xiaojiang Liu, Jianhao Yan, Deng Cai, Huayang Li, and Jian Li. 2022. Learning to break the loop: Analyzing and mitigating repetitions for neural text generation. *Advances in Neural Information Processing Systems* 35 (2022), 3082–3095.
[51] Lekang Yang, Yuetong Liu, Yitong Zhang, and Jia Li. 2025. DiffTester: Accelerating Unit Test Generation for Diffusion LLMs via Repetitive Pattern. *arXiv preprint arXiv:2509.24975* (2025).

[52] Ling Yang, Ye Tian, Bowen Li, Xinchen Zhang, Ke Shen, Yunhai Tong, and Mengdi Wang. 2025. Mmada: Multimodal large diffusion language models. *arXiv preprint arXiv:2505.15809* (2025).
[53] Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. 2025. Dream 7b: Diffusion large language models. *arXiv preprint arXiv:2508.15487* (2025).
[54] Yitong Zhang, Shiteng Lu, and Jia Li. 2026. Grammar-Constrained Decoding Can Jailbreak LLMs into Generating Malicious Code. *arXiv preprint arXiv:2606.11817* (2026).
[55] Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Lei Shen, Zihan Wang, Andi Wang, Yang Li, et al. 2023. Codegeex: A pre-trained model for code generation with multilingual benchmarking on humaneval-x. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5673–5684.
[56] Fengqi Zhu, Rongzhen Wang, Shen Nie, Xiaolu Zhang, Chunwei Wu, Jun Hu, Jun Zhou, Jianfei Chen, Yankai Lin, Ji-Rong Wen, et al. 2025. LLaDA 1.5: Variance-Reduced Preference Optimization for Large Language Diffusion Models. *arXiv preprint arXiv:2505.19223* (2025).
[57] Zijian Zhu, Fei Ren, Zhanhong Tan, and Kaisheng Ma. 2026. ES-dLLM: Efficient Inference for Diffusion Large Language Models by Early-Skipping. *arXiv preprint arXiv:2603.10088* (2026).

Received 2026-01-30; accepted 2026-06-25